

Digital Electronics

Circuit Example: Carry Lookahead

Adder

S Le Goff

June 2026

7. Carry Lookahead Adder

A carry lookahead adder (CLA) is a binary adder that is designed to operate faster than the ripple-carry adder (RCA) that was introduced earlier.

We remember that the RCA's critical path, that is the path that puts an upper limit of the operating speed of the circuit, corresponds to the computation of the final carry or, equivalently, the final sum bit.

The computation time of an n -bit RCA is given by $T_{RCA,n} \sim 2nt_p$, where t_p denotes the propagation time through a logic gate. For simplicity's sake, it has been assumed that all logic gates introduce the same delay. This is an oversimplification which can be overlooked for the time being but not for much longer than that. For instance, the computation time would be $T_{RCA,64} \sim 128t_p$ for a 64-bit RCA. That is considered to be far too slow for most applications.

We need to find some techniques to accelerate the computations of the carries, and especially that of the final one.

To this end, let us revisit the design of a full adder. And let us introduce two binary quantities $G = AB$ and $P = A \oplus B$. The meaning of these bits is as follows:

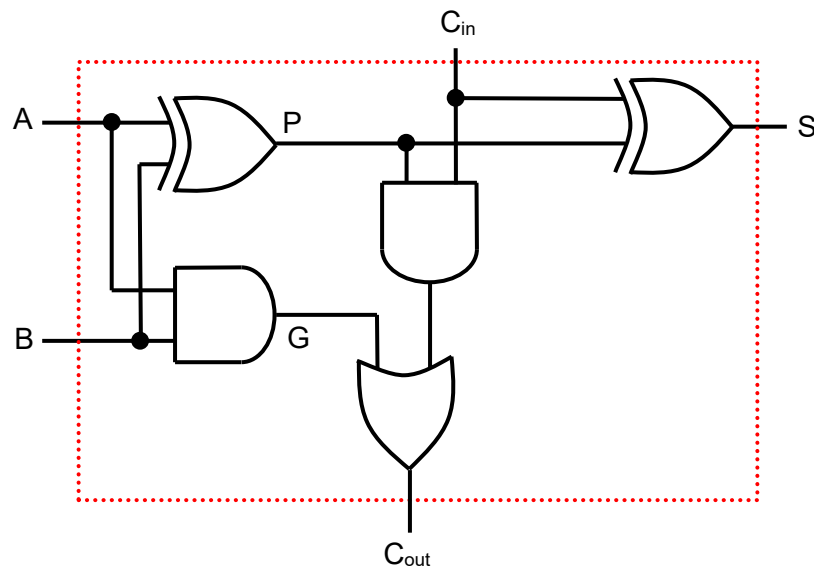
- (1) If $G = 1$, then an output carry $C_{out} = 1$ is produced by the full adder. regardless of the value of the input carry C_{in} . One can say that a carry is generated by this stage in this case.
- (2) If $P = 1$, then an output carry $C_{out} = 1$ is produced by this stage provided that the input carry was already equal to one, $C_{in} = 1$. In this case, the value of indicates that the stage does propagate an input carry but does not generate it.

To fully appreciate the meaning of the bits G and P for the carry computation logic circuit, consider the truth table shown below.

G		P	C _{in}	C _{out}
0		0	0	0
0		0	1	0
0		1	0	0
0		1	1	1
1		0	0	1
1		0	1	1
1		1	0	1
1		1	1	1

After simplification, we obtain the Boolean expression $C_{out} = G + PC_{in}$. This equation implies that “ $C_{out} = 1$ if ($G = 1$ OR ($P = 1$ AND $C_{in} = 1$)), a behaviour which exactly corresponds to the above definitions of G and P .

As for the expression of the sum bit S , we can now write $S = C_{in} \oplus (A \oplus B) = C_{in} \oplus P$. The logic diagram of this new version of the full adder is depicted below.



This structure is simpler than the full adder implementation introduced earlier since it makes use of a two-input OR gate instead of a three-input OR gate as well as two two-input AND gates instead of three two-input OR gates. In any case, we can cascade these full adders to design an n -bit adder and that would still be an RCA at the end of the day, with the same computation delay issue as the RCA studied earlier. That is not what we are going to do

hereafter. Instead, we are going to try to accelerate the carry computation by introducing some degree of parallelism in it.

Let us consider as an example the case of a four-bit adder that operates on two operands $A = (A_4A_3A_2A_1)$ and $B = (B_4B_3B_2B_1)$. The four carries are generated using the following set of equations:

- $C_{out,1} = G_1 + P_1C_{in}$;
- $C_{out,2} = G_2 + P_2C_{out,1}$;
- $C_{out,3} = G_3 + P_3C_{out,2}$;
- $C_{out,4} = G_4 + P_4C_{out,3}$.

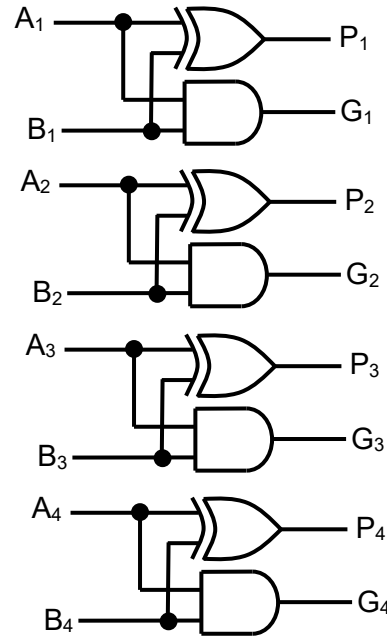
All quantities G_i and P_i , $i \in \{1, 2, 3, 4\}$, have been previously calculated in parallel.

By combining and developing these four equations, we can remove the dependencies between the successive carry bits:

- $C_{out,1} = G_1 + P_1C_{in}$;
- $C_{out,2} = G_2 + P_2C_{out,1} = G_2 + P_2(G_1 + P_1C_{in}) = G_2 + P_2G_1 + P_2P_1C_{in}$;
- $C_{out,3} = G_3 + P_3C_{out,2} = G_3 + P_3(G_2 + P_2G_1 + P_2P_1C_{in}) = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1C_{in}$
- $C_{out,4} = G_4 + P_4C_{out,3} = G_4 + P_4(G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1C_{in})$
 $\rightarrow C_{out,4} = G_4 + P_4G_3 + P_4P_3G_2 + P_4P_3P_2G_1 + P_4P_3P_2P_1C_{in}$.

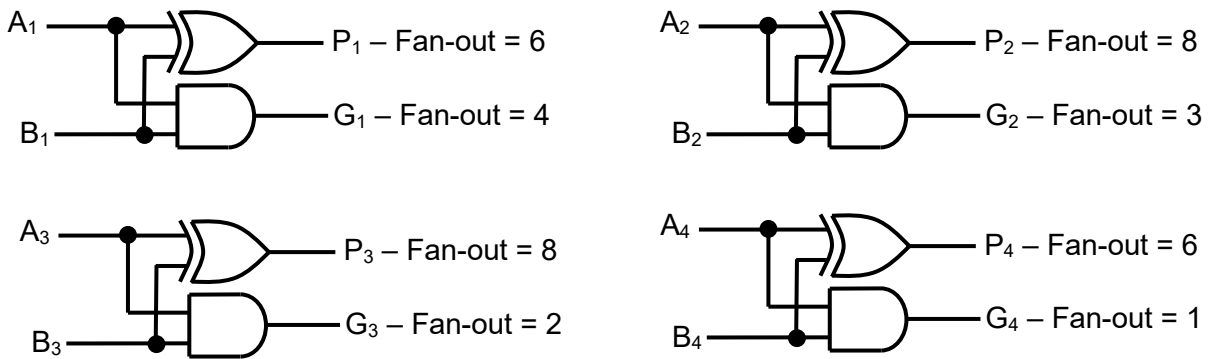
Based on those four Boolean expressions, we can design a carry-lookahead adder (CLA) than could operate faster than the traditional ripple-carry adder. A CLA is designed using three separate blocks.

The first of these blocks computes the bits G and P for all stages. Its logic diagram is shown below for the case $n = 4$.



All computations are done in parallel, and the number of logic layers is equal to one. The delay introduced by this block is equal to t_{p1} , where $t_{p,1}$ designates the propagation time of the slowest logic gate in this parallel structure. We must be aware of the following points:

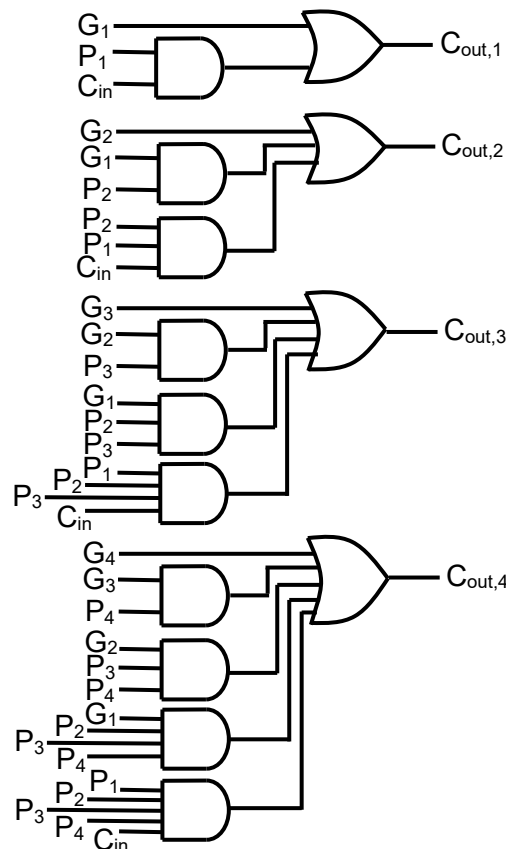
1. AND gates are inherently faster than XOR gates.
2. These eight gates have different capacitive loads, i.e. different fan-outs. Unfortunately, it turns out that the fan-outs can be quite large, especially for the XOR gates, as shown below.



To be on the safe side, we should assume that t_{p1} is associated with the slowest gate, i.e. $t_{p1} = t_p(\text{XOR, fanout} = 8)$. Or, we can even be more precise by considering the high-to-low and low-to-high output transitions separately: $t_{plh,1} = t_{plh}(\text{XOR, fanout} = 8)$ and $t_{phl,1} = t_{phl}(\text{XOR, fanout} = 8)$.

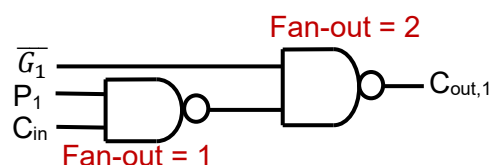
3. There are different ways to design XOR gates using transistors and these different implementations lead to different propagation times. But our investigations have shown that the differences between various implementations are not necessarily significant.

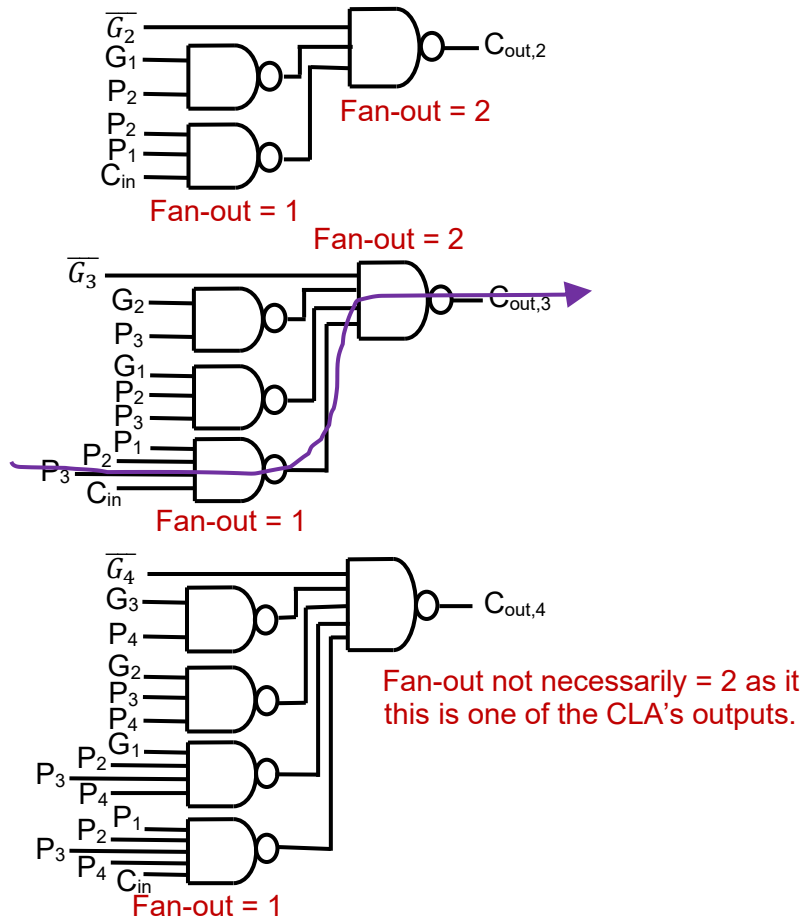
The second block in a CLA computes all the carry bits for all stages using the four Boolean expressions previously derived and the bits G and P computed by the first block. Its logic diagram is shown below for the case $n = 4$.



All computations are done in parallel, and the number of logic layers is equal to two. We can make the following comments:

1. This structure is in practice implemented using two layers of NAND gates, as shown below, because NAND gates are faster and simpler to build than both AND and OR gates.





The critical path through the CLA corresponds to the propagation time of a four-input NAND gate with a unit fan-out plus that of another four-input NAND gate with a fan-out of two, as indicated by the purple arrow on the above logic diagram.

The critical path does not pass through the five-input NAND gates because those gates generate the final carry-out $C_{out,4}$ of the CLA that does not need any further processing, whereas the four-input NAND gates compute the carry-out $C_{out,3}$ that, unlike $C_{out,4}$, will still have to go through a final XOR gate, thus adding more delay. This is will become perfectly clear later on once we compute actual propagation times for a CLA.

We can thus write expressions of the relevant propagation times, $t_{plh,2}$ and $t_{phl,2}$, through the CLA's second block:

- $t_{plh,2} = t_{plh}(4 - \text{input NAND, fanout} = 2) + t_{phl}(4 - \text{input NAND, fanout} = 1);$
- $t_{phl,2} = t_{phl}(4 - \text{input NAND, fanout} = 2) + t_{plh}(4 - \text{input NAND, fanout} = 1).$

2. It is observed that the circuit's structure is very irregular, which makes it difficult to design, much more so than the RCA that is, on the contrary, remarkable for its regularity.

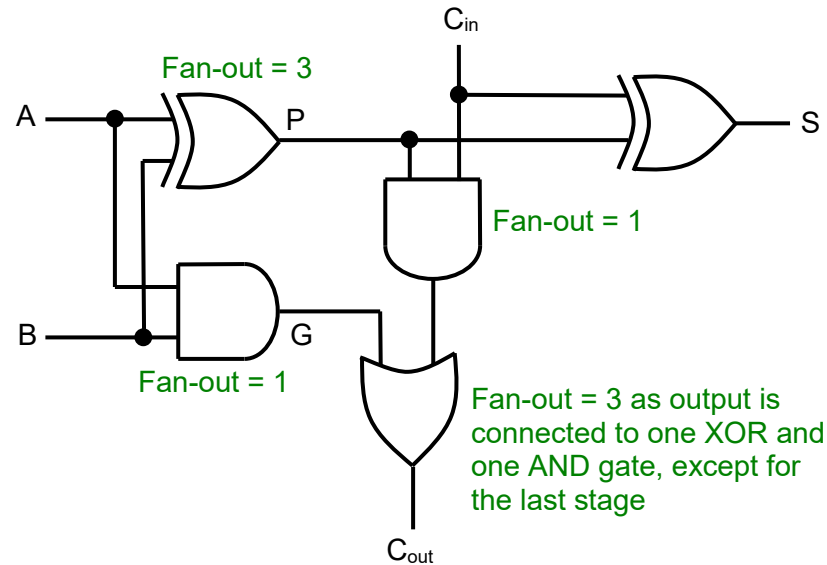
3. One can notice that some inputs drive a rather large number of gates, as indicated in the table below. This puts a high capacitive load on some of the first block's gates generating these inputs. Recall that the higher the capacitive load, represented by the parameter known as fan-out, the higher the propagation times.

Input	Number of times the input is used by the second block (case $n = 4$)
C_{in}	$1 \times n \text{ equations} = 4$
G_1	$1 \times n \text{ equations} = 4$
G_2	$1 \times (n-1) \text{ equations} = 3$
G_3	$1 \times (n-2) \text{ equations} = 2$
G_4	$1 \times (n-3) \text{ equations} = 1$
P_1	$1 \times n \text{ equations} = 4$
P_2	$2 \times (n-1) \text{ equations} = 6$
P_3	$3 \times (n-2) \text{ equations} = 6$
P_4	$4 \times (n-3) \text{ equations} = 4$

This is yet another issue that is much less present in an RCA because, in a regular cascade of full adders, the worst case that is encountered corresponds to a fan-out of four for the three-input OR gate generating each carry-out (except the last one).

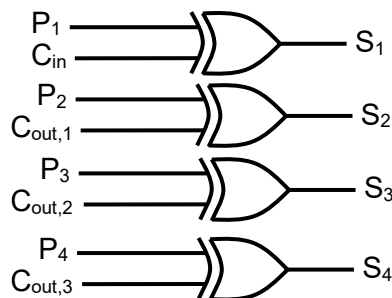
In the improved full adder version introduced a bit earlier, the worst-case scenario is even less penalising as it is associated with the carry-generating OR gate that has a fan-out of three as shown in the figure below.

Note that these fan-out numbers assume, for simplicity's sake, that all transistors in the circuit are minimally sized.



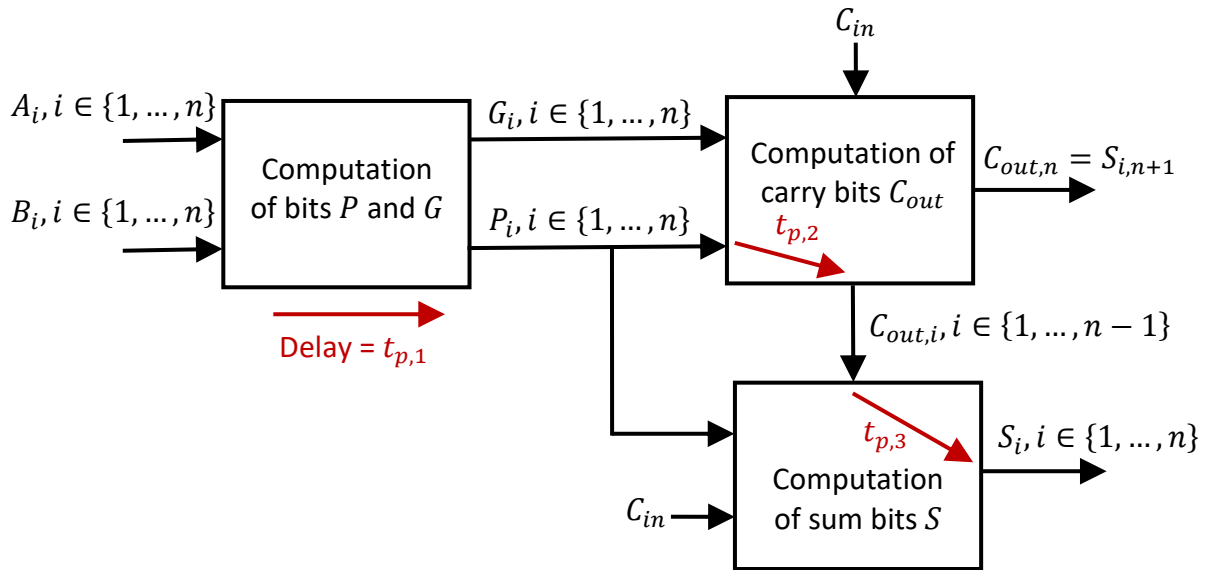
In summary, the carry's computation block suffers from major drawbacks that worsen, and quickly become impossible to overcome, as the operands' size increases beyond $n = 4$. That is the reason why the CLA's structure shown above is rarely used in practice for $n > 4$. The issue of designing carry's computation blocks suitable for the addition of large numbers ($n > 4$) will be addressed later on.

The third and final block in a CLA computes all the sum bits by XORing the quantities P_i , $i \in \{1, \dots, n\}$, generated by the first block as well as the carry bits C_{in} and $C_{out,i}$, $i \in \{1, \dots, n - 1\}$, produced by the second block. Its logic diagram is shown below for the case $n = 4$.



All computations are done in parallel, and the number of logic layers is clearly equal to one. Hence, the delay introduced by this block is denoted as $t_{p,3}$.

In summary. The diagram below depicts the architecture of a CLA, showing the three blocks previously described.



Is a CLA really faster than an RCA?

The literature says “yes, but not a lot faster in the case $n = 4$ ”. To me, the speed advantage of CLA over RCA is not so obvious and we will thus need to check it carefully.

To be certain, the only thing to do is to compute actual propagation times for equivalent RCA and CLA circuits to see which technique is performing best and by how much. This is exactly what we hereafter propose to do by considering a CMOS 45-nm process and simulating using the electric simulator LTspice, a free software kindly provided by Analog Devices Inc.

By running a set of simulations on an inverter and several NAND gates with various numbers of inputs and fan-out values, we have obtained the table given below that provides approximations of actual propagation times.

To measure a propagation time with accuracy, we found the following method to be the best: set the input reference so that it crosses the $V_{DD}/2$ level at a simple-to-remember time, such as for instance 10ps for the rise and 110ps for the fall, and then place the cursor on the output signal to read the corresponding time at which it also crosses the $V_{DD}/2$ level

directly. Finally, either subtract 10 or 110ps from this reading to obtain an accurate approximation for the corresponding propagation time.

In the table below, configurations 1 and 3 are worst-case scenarios and are thus to be considered by default. However, if we are certain that input signals settle at very different times, configurations 2 and 4 can be considered instead as they then become realistic and are less penalising. We would exploit the knowledge of the input signals' arrival times by connecting the early input to the NMOS closest to ground, and the late input to the NMOS closest to output node.

L = W = 45nm, input's rise and fall times = 5ps						
	Inverter	Two-input NAND gate	Three-input NAND gate	Four-input NAND gate	Five-input NAND gate	Input configuration
$\uparrow t_{p_{th}}$	FO = 1 → 3.9ps FO = 2 → 5.7ps FO = 3 → 7.7ps FO = 4 → 9.8ps FO = 6 → 14.5ps FO = 8 → 19.8ps	5.7ps 7.6ps 9.7ps 11.9ps 16.6ps 21.5ps	7.2ps 9.3ps 13.8ps	8.3ps 10.6ps	9.2ps 11.5ps	1
		4.4ps 6.2ps 8.3ps 10.4ps 15.1ps 20.2ps	4.9ps 6.7ps 11.0ps	5.4ps 7.3ps	5.9ps 7.7ps	2
$\downarrow t_{p_{hl}}$	FO = 1 → 2.7ps FO = 2 → 3.9ps FO = 3 → 5.1ps FO = 4 → 6.2ps FO = 6 → 8.6ps FO = 8 → 11.0ps	6.2ps 8.3ps 10.4ps 12.5ps 16.8ps 21.1ps	10.9ps 13.9ps 20.1ps	16.9ps 20.8ps	24.1ps 29.1ps	3
		5.0ps 7.0ps 9.1ps 11.2ps 15.6ps 20.0ps	7.7ps 10.4ps 16.6ps	9.9ps 13.9ps	12.5ps 17.5ps	4

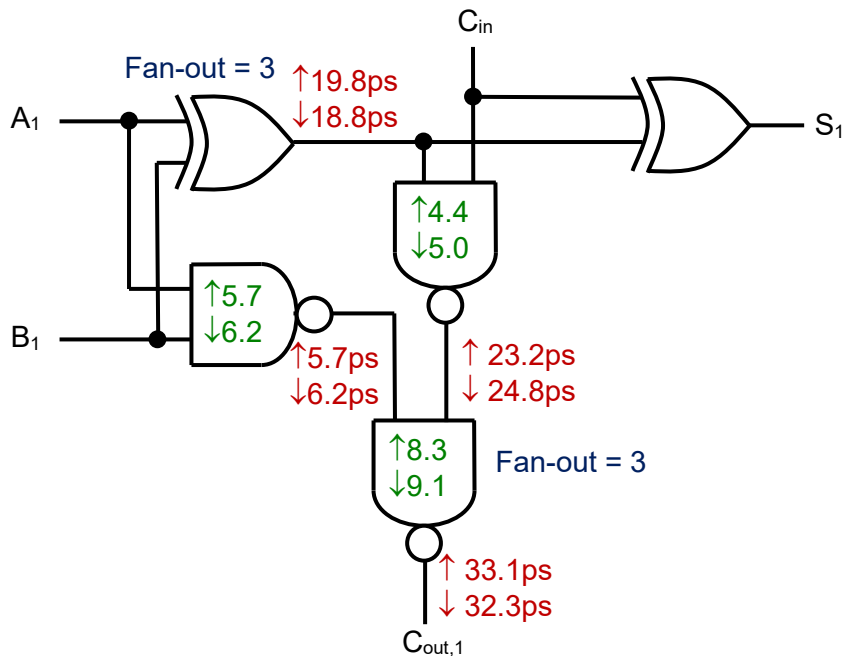
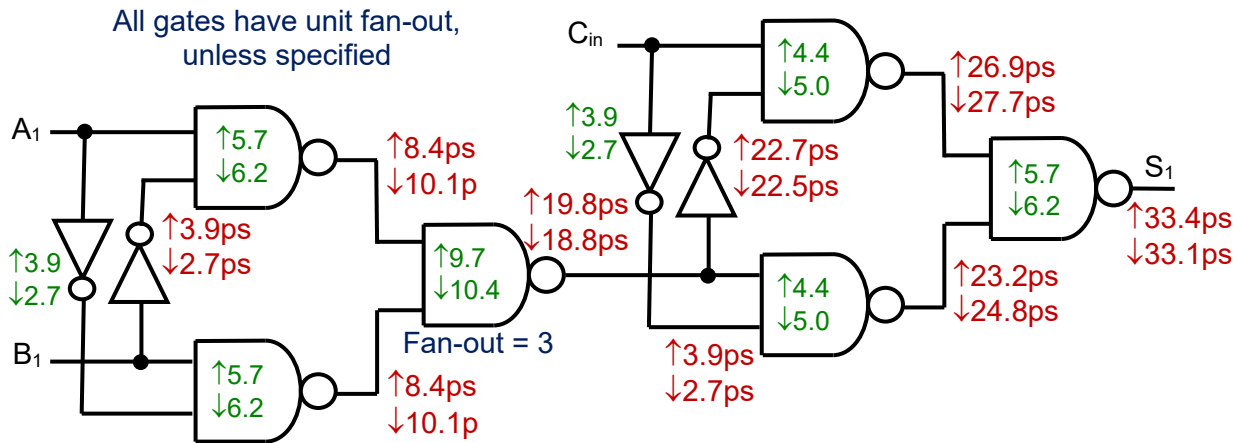
Configuration 1: All inputs = 1, except that of NMOS closest to ground which switches from 1 to 0.

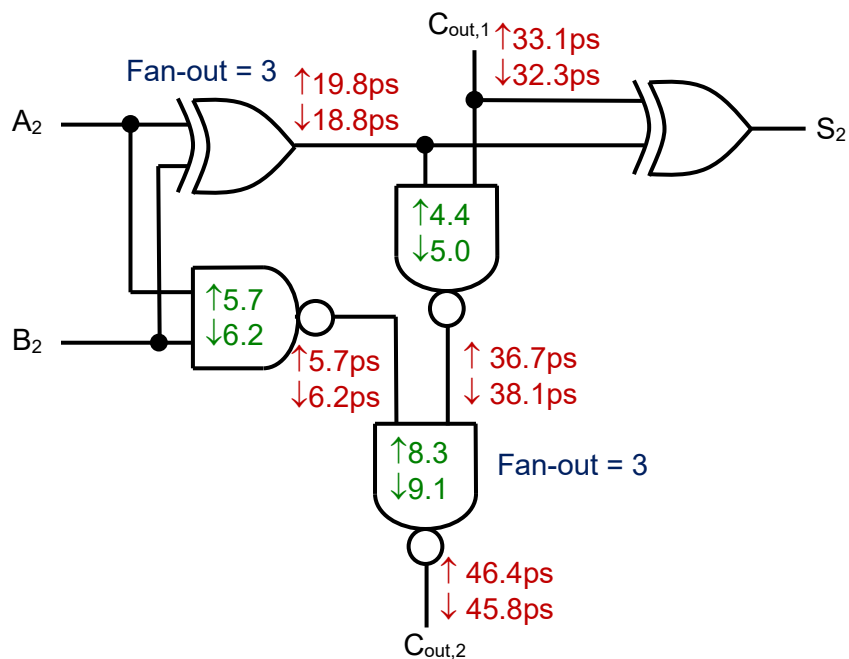
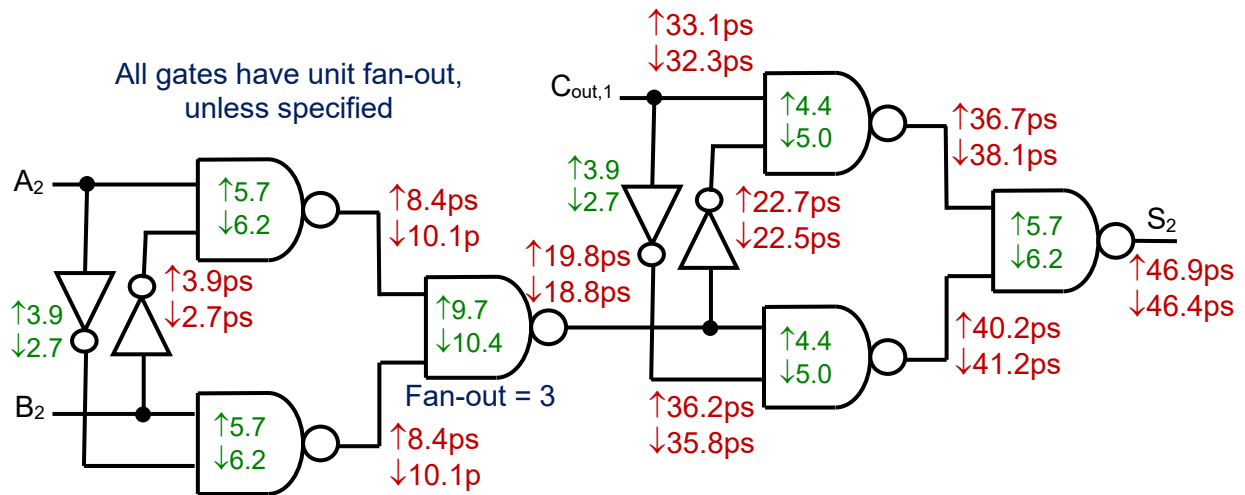
Configuration 2: All inputs = 1, except that of NMOS closest to output which switches from 1 to 0.

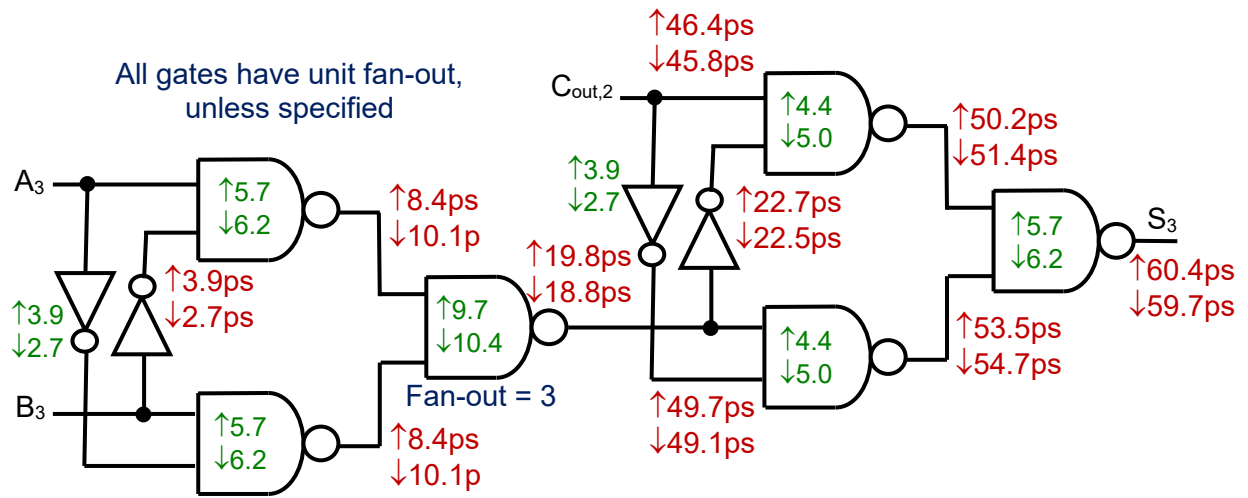
Configuration 3: All inputs = 1, except that of NMOS closest to ground which switches from 0 to 1.

Configuration 4: All inputs = 1, except that of NMOS closest to output which switches from 0 to 1.

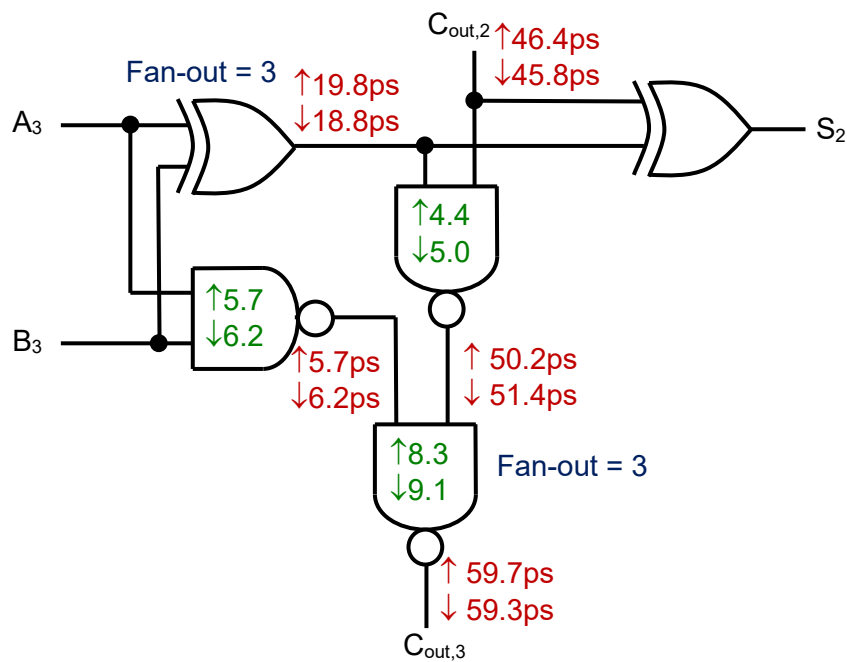
Computation time of a four-bit RCA based on the improved full adder version (two 2-input XOR gates + three 2-input NAND gates)



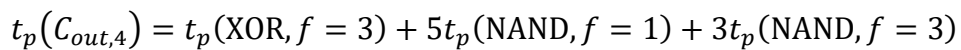
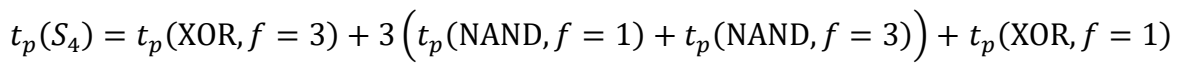




$$t_p(S_3) = t_p(\text{XOR}, f = 3) + 2 \left(t_p(\text{NAND}, f = 1) + t_p(\text{NAND}, f = 3) \right) + t_p(\text{XOR}, f = 1)$$



$$t_p(C_{out,3}) = t_p(\text{XOR}, f = 3) + 3 \left(t_p(\text{NAND}, f = 1) + t_p(\text{NAND}, f = 3) \right)$$



Output bit	Approximation of computation time for 4-bit RCA
Sum bit S_1	33.4ps
Sum bit S_2	46.9ps
Sum bit S_3	60.4ps
Sum bit S_4	73.9ps
Carry bit $C_{out,1}$	33.1ps
Carry bit $C_{out,2}$	46.4ps
Carry bit $C_{out,3}$	59.7ps
Carry bit $C_{out,4}$	73.0ps

The computation time $T_{RCA,n=4}$ of the four-bit RCA is thus equal to 73.9ps.

We can easily extend these results obtained for $n = 4$ bits to any arbitrary operand's size n . The general expression of the computation time is

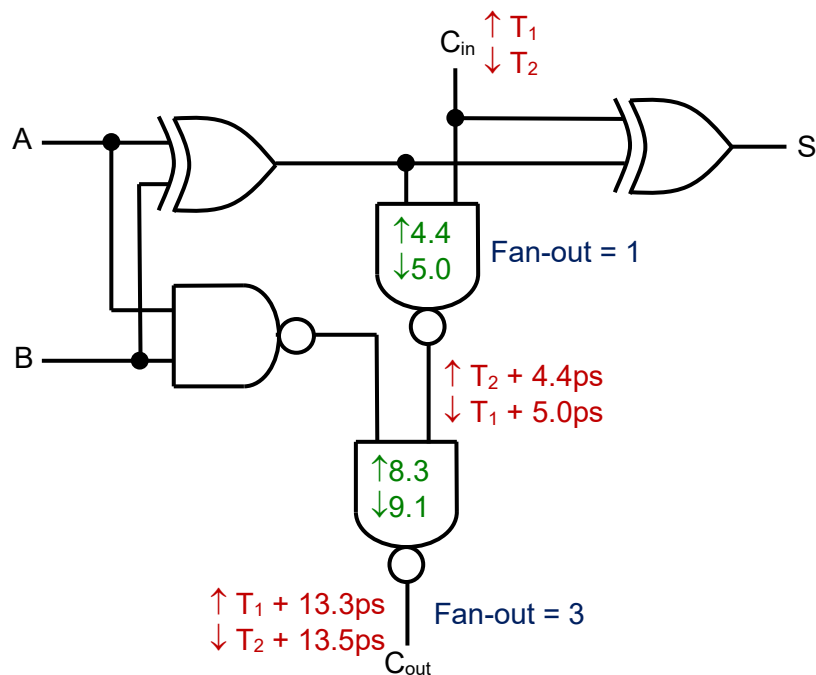
$$T_{RCA,n} = t_p(\text{XOR}, f = 3) + (n - 1) \cdot \left(t_p(\text{NAND}, f = 1) + t_p(\text{NAND}, f = 3) \right) + t_p(\text{XOR}, f = 1),$$

with

$$t_p(\text{XOR}, f = 3) + t_p(\text{XOR}, f = 1) = 33.4ps$$

and

$$t_p(\text{NAND}, f = 1) + t_p(\text{NAND}, f = 3) = 13.5ps.$$

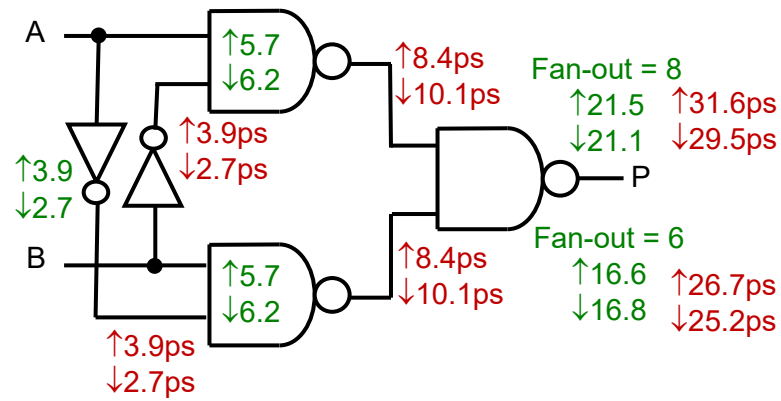


Operand size	Computation time for n -bit RCA
$n = 16$	236ps
$n = 32$	452ps
$n = 64$	884ps

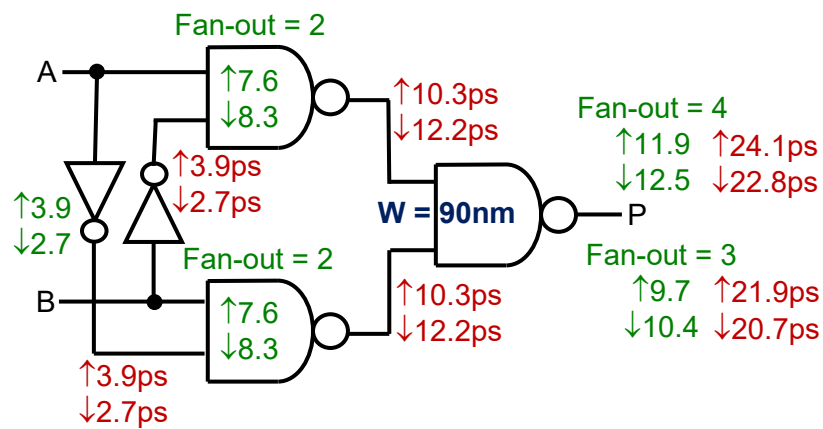
Computation time of a four-bit CLA

L = W = 45nm, input's rise and fall times = 5ps						
	Inverter	Two-input NAND gate	Three-input NAND gate	Four-input NAND gate	Five-input NAND gate	Input configuration
$\uparrow t_{plh}$	FO = 1 → 3.9ps FO = 2 → 5.7ps FO = 3 → 7.7ps FO = 4 → 9.8ps FO = 6 → 14.5ps FO = 8 → 19.8ps	5.7ps 7.6ps 9.7ps 11.9ps 16.6ps 21.5ps	7.2ps 9.3ps 13.8ps	8.3ps 10.6ps	9.2ps 11.5ps	1
		4.4ps 6.2ps 8.3ps 10.4ps 15.1ps 20.2ps	4.9ps 6.7ps 11.0ps	5.4ps 7.3ps	5.9ps 7.7ps	2
$\downarrow t_{phl}$	FO = 1 → 2.7ps FO = 2 → 3.9ps FO = 3 → 5.1ps FO = 4 → 6.2ps FO = 6 → 8.6ps FO = 8 → 11.0ps	6.2ps 8.3ps 10.4ps 12.5ps 16.8ps 21.1ps	10.9ps 13.9ps 20.1ps	16.9ps 20.8ps	24.1ps 29.1ps	3
		5.0ps 7.0ps 9.1ps 11.2ps 15.6ps 20.0ps	7.7ps 10.4ps 16.6ps	9.9ps 13.9ps	12.5ps 17.5ps	4

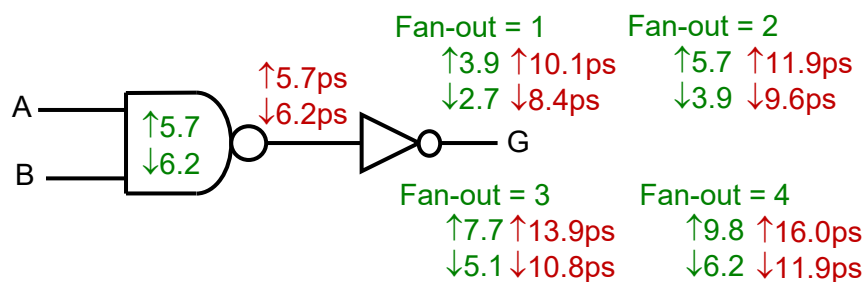
Block 1: Computation of propagate bits $P = A \oplus B = A\bar{B} + \bar{A}B$



A smarter design leads to improved propagation times in the presence of high fan-out values:



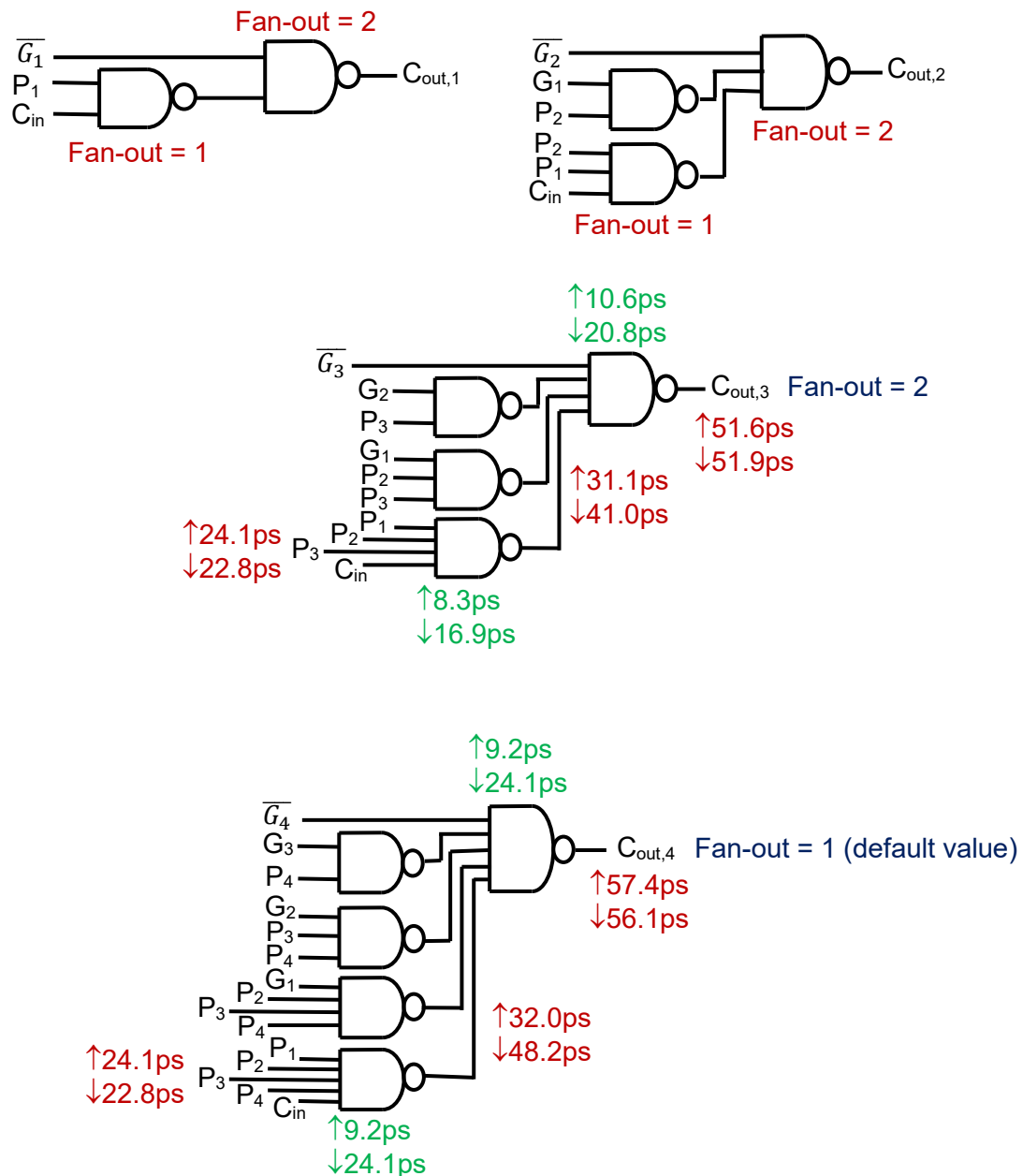
Block 1: Computation of generate bits $G = AB$



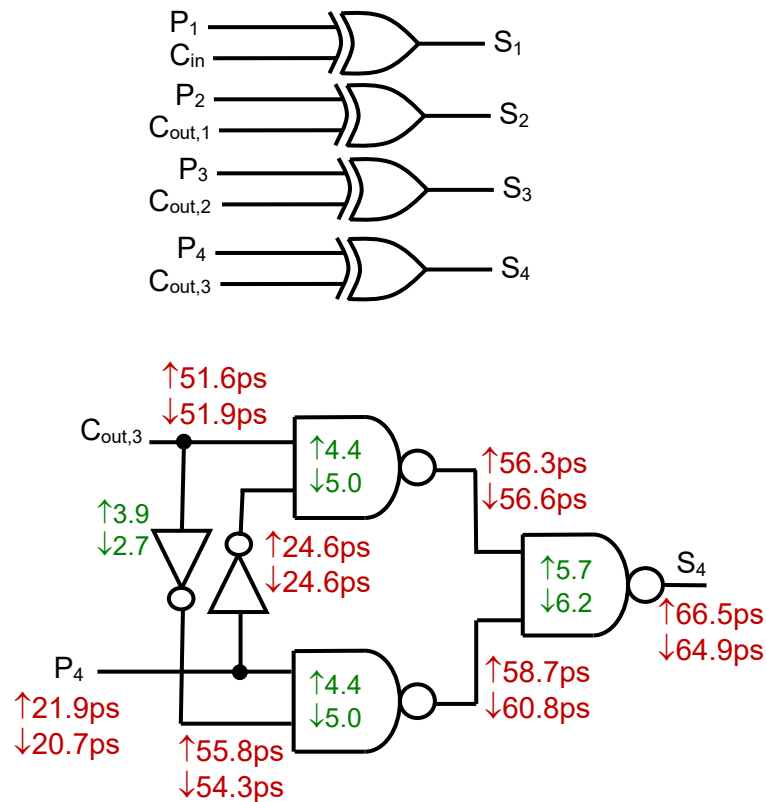
Output	Fan-out	Computation time
G_1	4	$\uparrow 16.0\text{ps}$ $\downarrow 11.9\text{ps}$
G_2	3	$\uparrow 13.9\text{ps}$ $\downarrow 10.8\text{ps}$
G_3	2	$\uparrow 11.9\text{ps}$ $\downarrow 9.6\text{ps}$
G_4	1	$\uparrow 10.1\text{ps}$ $\downarrow 8.4\text{ps}$

P_1	6	$\uparrow 21.9\text{ps} \downarrow 20.7\text{ps}$
P_2	8	$\uparrow 24.1\text{ps} \downarrow 22.8\text{ps}$
P_3	8	$\uparrow 24.1\text{ps} \downarrow 22.8\text{ps}$
P_4	6	$\uparrow 21.9\text{ps} \downarrow 20.7\text{ps}$

Block 2: Computation of carry bits C_{out}



Block 3: Computation of sum bits S



Output bit	Approximation of computation time for 4-bit RCA	Approximation of computation time for 4-bit CLA
Sum bit S_1	33.4ps	Not computed
Sum bit S_2	46.9ps	Not computed
Sum bit S_3	60.4ps	Not computed
Sum bit S_4	73.9ps	66.5ps
Carry bit $C_{out,1}$	33.1ps	Not computed
Carry bit $C_{out,2}$	46.4ps	Not computed
Carry bit $C_{out,3}$	59.7ps	51.9ps
Carry bit $C_{out,4}$	73.0ps	57.4ps

Conclusion: The four-bit CLA performs $73.9\text{ps} - 66.5\text{ps} = 7.4\text{ps}$ faster than an equivalent RCA, based on LTspice simulation numbers assuming a 45-nm CMOS technology. This corresponds to a 10% reduction in computation time. Does the added complexity of a CLA truly justify its use for a meagre 10% improvement in computation time with respect to the RCA?

We must, in any case, keep in mind that four-bit adders are often not very useful in practice since, for most applications, the size of the operands is often as high as $n = 32$ or 64 . The CLA architecture studied here cannot be extended to such sizes because doing so would require the utilization of NAND gates with an impractical number of inputs, e.g. 65 inputs in the case $n = 64$. In addition to this, some fan-out values would be very high, further complicating the design process.

The CLA architecture does however possess some key potential over the RCA approach owing to its parallel structure. One should therefore attempt to improve it by making it more regular, making it more scalable with the operands' size, reducing the fan-outs of some logic gates, and completely avoiding the use of gates with a large number of inputs.

In fact, one should focus one's efforts on the second block that computes the carry bits, as the first and third blocks are already clearly optimal.